SAVONIA

THIS IS THE WAY TO CHANGE THE WORLD.

PROJECT REPORT – BACHELOR'S DEGREE
TECHNOLOGY, COMMUNICATION AND TRANSPORT

# SAVONIA-HONEYPOT


AUTHOR/S:

**Min Set Ko - Project Lead**

**Md Mohiuddin Mollah - Documentation Lead, Honeypot Implementation Support**

**Arsenii Marchenko - Adversarial Simulation Lead & Product Assembly**

**Margarita Filenko – 3d Modelling & Hardware Enclosure Design**

**Joonas Tahvanainen - Honeypot Architect, Detection & Analysis Lead**

**Rupashri Das - Web Server Implementation**

SAVONIA UNIVERSITY OF APPLIED SCIENCES

THESIS
Abstract

| | |
|---|---|
| Field of Study<br>Technology, Communication and Transport | |
| Degree Programme<br>Degree Programme in Internet of Things | |
| Author<br>Md Mohiuddin Mollah, Min Set Ko | |
| Title of Project<br>IoT Honeypot | |
| Date 05.12.2025 | Pages/Appendices 15 |
| Client Organization /Partners<br>Savonia UAS | |


Abstract

To address the critical gap between theoretical cybersecurity concepts and practical application in higher education, this report presents the development of a comprehensive, all-in-one honeypot educational platform. Designed to provide students and training institutions with a controlled environment for mastering Intrusion Detection Systems (IDS) and honeypot technologies, the project involved the design and implementation of a low-interaction IoT lab on Raspberry Pi 5 devices. The architecture establishes an isolated subnet managed by an OpenWrt router, housing a dedicated attacking machine for offensive simulations and a multi-layered defensive environment.

The core system integrates a Cowrie honeypot for SSH/Telnet emulation, a Suricata IDS for real-time threat monitoring, and a Nginx web server hosting a static corporate interface with a deliberate PHP vulnerability. For advanced observability, the platform employs a Promtail–Loki–Grafana stack to facilitate centralized log collection and visualization. To evaluate the environment, a red-team vs. blue-team exercise was conducted; the red team executed scanning, brute-force attempts, and web exploits, while the blue team monitored alerts via EveBox and Grafana. Results confirm that the system successfully recorded and visualized various attack vectors, demonstrating that this compact, low-interaction lab effectively models attacker behaviors in an IoT context while significantly enhancing learner proficiency in Linux administration, network routing, and security monitoring.



Keywords: Blue Team, Red Team, Honeypot, Cowrie, IoT, Network security, Cyber security, Penetration testing, IDS, Suricata, Loki, Grafana, Eve Box, NGINX, PHP, Python, Linux,

1 INTRODUCTION ..... 4
2 WHAT IS HONEYPOT? ..... 5
3 PROJECT WORKFLOW AND IMPLEMENTATION STEPS ..... 6
3.1 Network topology ..... 6
3.2 Subnet Creation with OpenWrt ..... 8
3.3 HoneyPi Deployment ..... 9
3.4 Web server Deployment ..... 9
3.5 Attacking Machine Deployment ..... 10
3.6 Red Team Vs Blue Team ..... 11
3.7 Results and observations ..... **Error! Bookmark not defined.**
4 HARDWARE AND SOFTWARE USED ..... 13
4.1 Raspberry Pi 5 ..... 13
4.2 Red -team tools ..... 13
4.3 Blue-team tool ..... 14
4.4 Supporting Infrastructure ..... 15
5 SECURITY CONSIDERATIONS AND THREAT LANDSCAPE ..... 16
6 FUTURE IMPLEMENTATION ..... 17
7 CONCLUSION ..... 19
8 REFERENCES ..... 20

# 1 INTRODUCTION

Internet of Things (IoT) devices are highly susceptible to cyberattacks, largely because many are deployed with weak default credentials, outdated firmware, and insecure network services. Beyond the devices themselves, it is equally important to protect mission-critical servers and edge gateways that process IoT sensor data. A successful denial-of-service (DoS) attack could disrupt these servers and halt essential data processing, leading to financial losses and operational downtime. Furthermore, credential-harvesting attacks may expose sensitive information, which adversaries can later leverage in phishing campaigns or ransomware operations.

The urgency of this issue is highlighted by the device search engine Shodan, which reveals millions of IoT devices exposed on the internet without proper protection. Vulnerability scanning of such devices has become a frontline battle between defenders and attackers. However, a significant challenge in addressing these threats is the gap between theoretical cybersecurity concepts taught in higher education and the practical application required in the field. Students often lack safe, controlled environments to experience real-time intrusion detection and honeypot mechanics.

To address this educational gap, the primary objective of this project is the development of a comprehensive, all-in-one honeypot platform designed for university students and training institutions. The system is implemented as a low-interaction IoT lab running on **Raspberry Pi 5** devices. This architecture features an isolated subnet managed by an **OpenWrt** router, ensuring complete separation from the institutional production network. The core environment integrates multiple security technologies: a **Cowrie honeypot** for SSH/Telnet emulation, a **Suricata Intrusion Detection System (IDS)** for threat monitoring, and an **Nginx** web server hosting a static corporate interface with a deliberate **PHP vulnerability** to demonstrate web application exploits.

To evaluate the system's effectiveness, an internal Red-Team vs. Blue-Team exercise was conducted. The Red Team simulated adversarial behaviors—including port scanning, brute-force attacks, and web exploits—using custom scripts and **Nmap**. In parallel, the Blue Team utilized a **Promtail–Loki–Grafana (PLG)** stack to monitor logs, analyze alerts via **EveBox**, and visualize attack patterns. This approach not only validates the technical capability of the honeypot to capture and display data but also allows participants to practice essential skills in defensive monitoring, log analysis, and incident reporting.

# 2 WHAT IS HONEYPOT?

A honeypot is **a decoy system** or service designed to mimic legitimate targets of cyberattacks, with the goal of attracting, detecting and analyzing malicious activity. It acts as a trap for attackers, allowing defenders to observe their tactics, techniques and procedures (TTPs) without risking production systems. Honeypots can emulate vulnerable services such as FTP, Telnet, SSH or web servers, and are often configured to look like high-value targets.

Honeypots are typically deployed in isolated or segmented network zones, such as a DMZ or a dedicated subnet, to prevent attackers from pivoting into real systems. This placement ensures that even if the honeypot is compromised, the attacker cannot access sensitive infrastructure. In internal networks, honeypots may also be placed near critical assets to detect lateral movement or insider threats.

Honeypots are broadly categorized by their level of engagement. Low-interaction honeypots simulate a limited set of services to lure attackers, whereas high-interaction honeypots run real operating systems and applications to capture deeper behavioral data. While high-interaction systems offer richer insights, they require significant hardware resources and rigorous isolation to prevent actual compromise.

Given the hardware constraints of the Raspberry Pi and the need for a secure, deployable solution, we utilized **Cowrie**, a low-interaction honeypot that emulates SSH and Telnet services. Cowrie minimizes risk by isolating the attacker in a fake shell environment. To augment this low-interaction setup and capture a broader spectrum of network threats, we integrated **Suricata IDS**. This combination allows us to maintain a lightweight footprint while simultaneously monitoring traffic signatures and logging shell interactions.

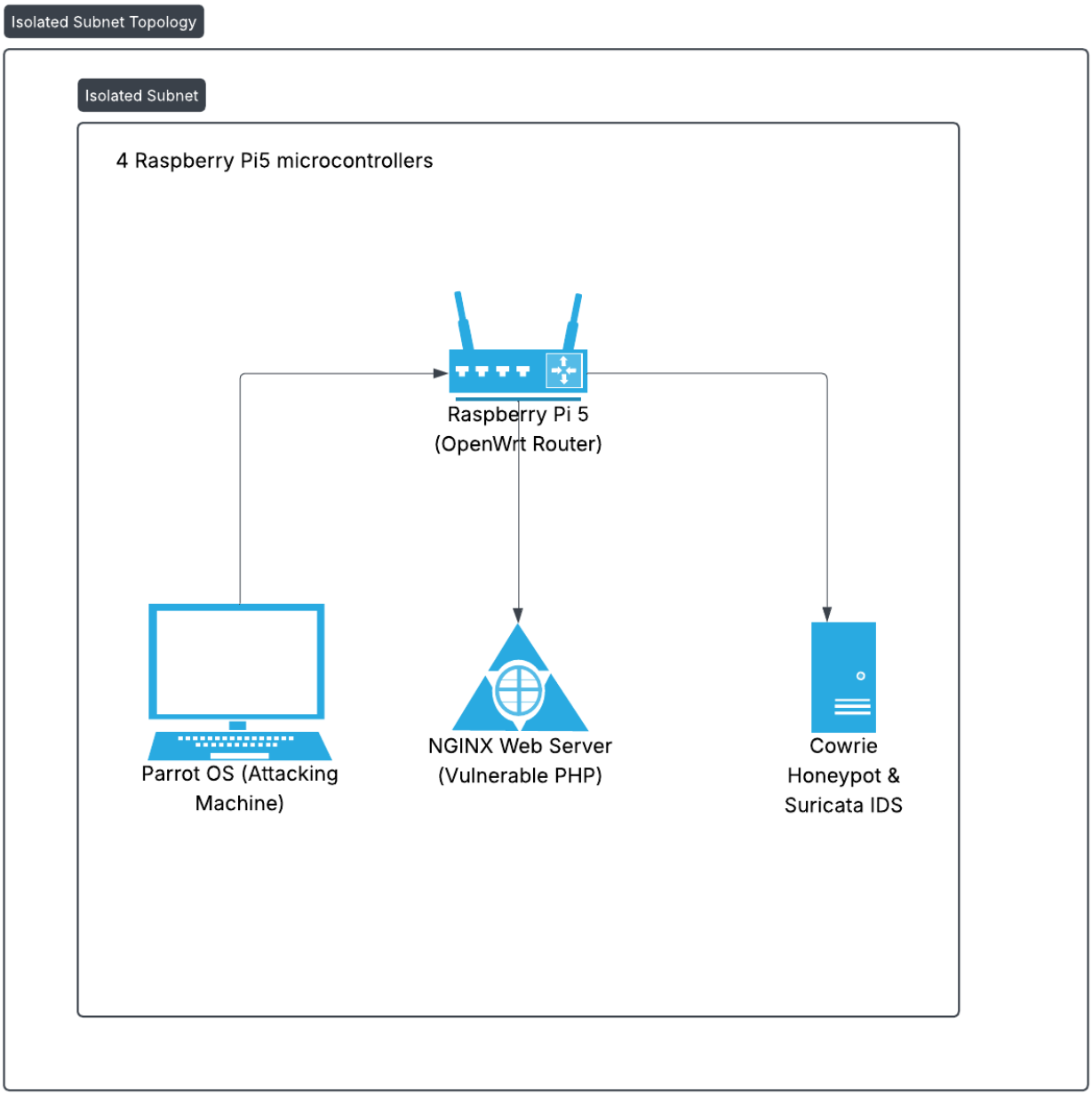
# 3 PROJECT WORKFLOW AND IMPLEMENTATION STEPS

## 3.1 Network topology

The overall network topology is built entirely on Raspberry Pi 5 devices. One Raspberry Pi runs OpenWrt and acts as the central router for an isolated wireless subnet. Other Raspberry Pi serves as the honeypot system, the attacker machine, and a reference web server.

The OpenWrt router creates an internal subnet using the address range 192.168.1.0/24. Its WAN interface connects towards the institutional network through a WAN firewall, while the LAN side is protected by a separate firewall and offers Wi-Fi access to lab devices. During red-team vs. blue-team simulations, the WAN cable is physically disconnected, so all attack traffic remains inside the lab environment and does not affect external networks.

The experimental architecture comprised four Raspberry Pi 5 devices. The first device was configured with OpenWrt to act as the gateway for the isolated subnet. The second device, running Parrot OS, operated as the adversarial node. The third device hosted a NGINX web server containing a static interface and an intentional PHP vulnerability. The fourth device functioned as the primary defensive node, hosting the Cowrie honeypot, Suricata IDS, and the logging infrastructure. To ensure containment, all components were restricted to the isolated network segment.


Isolated Subnet Topology
Isolated Subnet
4 Raspberry Pi5 microcontrollers
Raspberry Pi 5
(OpenWrt Router)
Parrot OS (Attacking
Machine)
NGINX Web Server
(Vulnerable PHP)
Cowrie
Honeypot &
Suricata IDS

## 3.2 Subnet Creation with OpenWrt

For this project, we utilized OpenWrt, a free and open-source router operating system, to transform a Raspberry Pi 5 into a fully functional network router. OpenWrt was selected for its flexibility, lightweight footprint and strong community support, making it ideal for custom lab environments.

The initial router setup was performed via command-line interface (CLI). During configuration, we encountered challenges with the **eth0** interface, which OpenWrt typically reserves for downstream management access to the web interface. For our lab topology, we needed eth0 to act as an upstream DHCP client to obtain connectivity from the Savonia network.

After reconfiguring the interface, we successfully repurposed eth0 to acquire an IP address via DHCP and provide **NAT-based internet access** to our internal subnet. This allowed honeypot, attacking machine, and monitoring tools to communicate externally when needed.

The **wlan0 interface**, paired with OpenWrt's bridge configuration and the Raspberry Pi's onboard radio chip, was configured to operate as a **wireless access point (AP)**. It also functions as a **DHCP server**, dynamically assigning IP addresses to any device connecting to the lab network. This setup supports flexible device onboarding and segmentation.

Our internal subnet is defined as **192.168.1.0/24**, a standard Class C private network range suitable for penetration testing exercises. To ensure isolation during **red team vs. blue team simulations**, we physically disconnect the eth0 cable, rendering the subnet offline and preventing any impact on the broader institutional network. For remote management or internet connectivity, eth0 can be reconnected to restore upstream connectivity.

This modular setup allows us to toggle between **offline simulation mode** and **online management mode**, ensuring both operational flexibility and network safety.

## 3.3 HoneyPi Deployment

For this project, we initially studied the HoneyPi framework and used it as a reference for designing a low-interaction honeypot on Raspberry Pi. In the final implementation, we built the honeypot stack manually on Raspberry Pi OS using **Cowrie**, a medium-interaction SSH/Telnet honeypot. Cowrie emulates a fake Linux file system and command shell and records every login attempt and command issued by an attacker. We control the service with the commands “cowrie start”, “cowrie stop” and “cowrie status” inside a Python virtual environment.

To gather actionable intelligence on common attack vectors, the **low-interaction Cowrie honeypot** on the fourth Raspberry Pi simulates vulnerable login services, exposing **SSH (port 22)** and **Telnet (port 23)** to lure attackers into a controlled environment. In our tests, Nmap also reported services on ports 111/tcp and 2222/tcp, which provided additional realism for the scanning phase. The honeypot is reachable only from the isolated subnet created by the OpenWrt router.

To make monitoring easier for the blue team, we connected Cowrie to a small observability stack running in Docker containers on the same Raspberry Pi. Cowrie’s log files are picked up by **Promtail**, forwarded to **Loki** for storage and then visualized in **Grafana** dashboards (port 3000). In this way, the blue team can either read raw Cowrie logs on the honeypot or view aggregated charts such as the number of logins attempts per session.

All these components operate completely offline inside the lab subnet. No honeypot services are exposed to the public internet, and the subnet is physically separated from the Savonia learning network.

## 3.4 Web server Deployment

One Raspberry Pi 5 was configured as a **web-server node** running a lightweight Ubuntu-based system with **NGINX**. The web server hosts static demo sites for fictional companies and a deliberately vulnerable PHP page. Static content makes the network look more like a real small company environment, while the vulnerable PHP application is used in the red-team scenario to demonstrate a simple web-application attack.

A custom Python script (console.py) running on the attacker machine simulates an exploit against this PHP page, based on a published PHP vulnerability (CVE/CWE). The goal is not to build a fully weaponized exploit but to generate realistic HTTP traffic and log entries that can be analyzed by the blue team and detected by the IDS.

## 3.5 Attacking Machine Deployment

A separate Raspberry Pi 5 was configured as the **attacking machine**. We started with a clean Parrot OS image tailored for Raspberry Pi 5 and installed additional firmware components to enable wireless monitor mode and packet injection. This allows common wireless security tools to run directly on the onboard Wi-Fi chipset without external adapters.

We chose Parrot OS instead of Kali Linux because it is less resource-intensive and offers smoother desktop experience on ARM-based devices. Parrot still includes many penetration-testing and privacy-focused tools, making it a good balance between usability and functionality on Raspberry Pi.

From this machine, the red team could perform host discovery, service scanning, password-guessing attempts, and controlled wireless-security tests against the lab's own access point.

On top of the standard tools provided by Parrot OS, we developed three small Python scripts to automate typical attacker steps:

1. **portscanner.py** – a custom port-scanner that goes through a range of IP addresses and ports, using socket connections and Nmap to find open services in the isolated subnet.
2. **testCow.py** – a password-brute-force script that repeatedly tries different username/password combinations against Cowrie's SSH/Telnet service to generate controlled login-failure patterns.
3. **console.py** – a proof-of-concept script that interacts with the vulnerable PHP page on the NGINX web server to simulate exploitation of a known PHP vulnerability.

These scripts, together with configuration files, are stored in the Red Team folder of our GitHub repository and are used only inside the isolated lab network for educational purposes.

## 3.6 Red Team Vs Blue Team

To gather actionable threat intelligence, we deployed a **low-interaction Cowrie honeypot** on the fourth Raspberry Pi, simulating vulnerable **SSH (port 22)** and **Telnet (port 23)** services to lure attackers.

To validate the lab's detection capabilities, we conducted a concluding **Red Team vs. Blue Team exercise**:

**Red Team Operations (Attack):**

**Reconnaissance:** The team utilized **Nmap** to map the OpenWrt subnet and **Portscanner.py** to target the honeypot. This revealed open ports 22 (SSH), 23 (Telnet), 111 (rpcbind), and 2222 (EtherNetIP-1), confirming the device as a management target.

**Service Attacks:** Using the `testcow.py` script, the team executed controlled brute-force attacks using short wordlists of common IoT credentials (e.g., "admin/password"). This generated specific failure sequences for **Cowrie, Loki, and Grafana** to visualize.

**Web Attacks:** Separately, the team targeted the web-server Raspberry Pi using `console.py`, sending crafted HTTP requests to the vulnerable PHP service to simulate the exploitation of a published vulnerability (CVE/CWE).

**Blue Team Operations (Defense):**

**Network Detection: Suricata IDS** monitored network traffic on the honeypot node, forwarding alerts regarding port scans and brute-force patterns to **EveBox**.

**Analysis:** The Blue Team monitored the **HoneyPi** live terminal, reviewed the PHP service logs, and verified email summaries.

**Correlation:** By comparing Red Team timestamps with system logs, the Blue Team successfully correlated the command execution sequence—from initial reconnaissance to active exploitation—validating the lab's monitoring infrastructure.

## 3.7 Results and observations

The exercise confirmed that a low-interaction honeypot can still provide valuable insight into attacker behavior. During each red-team session, the honeypot recorded sequences of port scans followed by concentrated Telnet login attempts. The logs showed repeated use of default-style usernames and simple passwords, which reflects common attack patterns seen against IoT devices on the internet.

In addition to the textual log files, the observability components confirmed that the pipeline worked as designed. Cowrie's raw logs showed repeated failed logins and command attempts, while Promtail successfully forwarded these events into Loki. In **Grafana**, the blue team could see dashboards summarizing the number of connection attempts, login failures, and other events over time. At the same time, **Suricata** generated alerts for port-scanning and suspicious traffic patterns, which were reviewed through the **EveBox** web interface. This combination of logs and dashboards made it easier to explain the attacks to other students and teachers

For this project phase, the status of the honeypot services was:

- **Port-scan logging:** active and functioning correctly
- **Telnet and SSH:** active; multiple failed login attempts captured. Successful connection with correct username and password combination

The logging and notification pipeline worked as intended. Real-time logs on the HoneyPi device gave immediate feedback during attacks, while the dashboard summaries provided evidence for later analysis and reporting. The portable enclosure with four Raspberry Pi 5 devices made it easy to move the whole lab and demonstrate it to others.

# 4 HARDWARE AND SOFTWARE USED

## 4.1 Raspberry Pi 5

The entire environment was built on **four Raspberry Pi 5 boards**, each dedicated to a specific role:

• Raspberry Pi 5 running **OpenWrt** as the router and wireless access point for the isolated subnet

• Raspberry Pi 5 running **Raspberry Pi OS** with the Cowrie honeypot and the Prom tail–Loki–Grafana–Suricata–Eve Box stack

• Raspberry Pi 5 running **Parrot OS** as the attacker machine with Nmap and custom Python scripts

• Raspberry Pi 5 running a lightweight **Ubuntu/NGINX web server** with static demo sites as well as deliberately vulnerable PHP page

This approach kept costs low while offering enough performance for multiple operating systems and services, and it allowed us to keep every function logically separated

## 4.2 Red-team tools

The red team mainly used:

• **Nmap** and the custom portscanner.py script to discover hosts and open ports

• **testcow.py** to perform controlled password-guessing attempts against Cowrie's SSH/Telnet service with small, controlled wordlists

• **console.py** to interact with and exploit the deliberately vulnerable PHP page on the NGINX web server

• **Hydra** and other standard Parrot OS tools for additional login-brute-force testing when needed All tools were used only inside the isolated lab subnet and for educational purposes

## 4.3 Blue-team tool

The Blue Team's toolkit included:

• Cowrie's built-in logging interface on the honeypot Raspberry Pi

• The **Promtail–Loki–Grafana** stack for collecting Cowrie logs and visualizing them in dashboards (Grafana on port 3000)

• **Suricata IDS** for detecting suspicious network traffic, with events reviewed through the **EveBox** web dashboard

• A dedicated email account for automatic log summaries (where needed)

• Standard Linux tools (for example nano, less, grep) for viewing and filtering log files

• Shared documentation files for recording observations during exercises

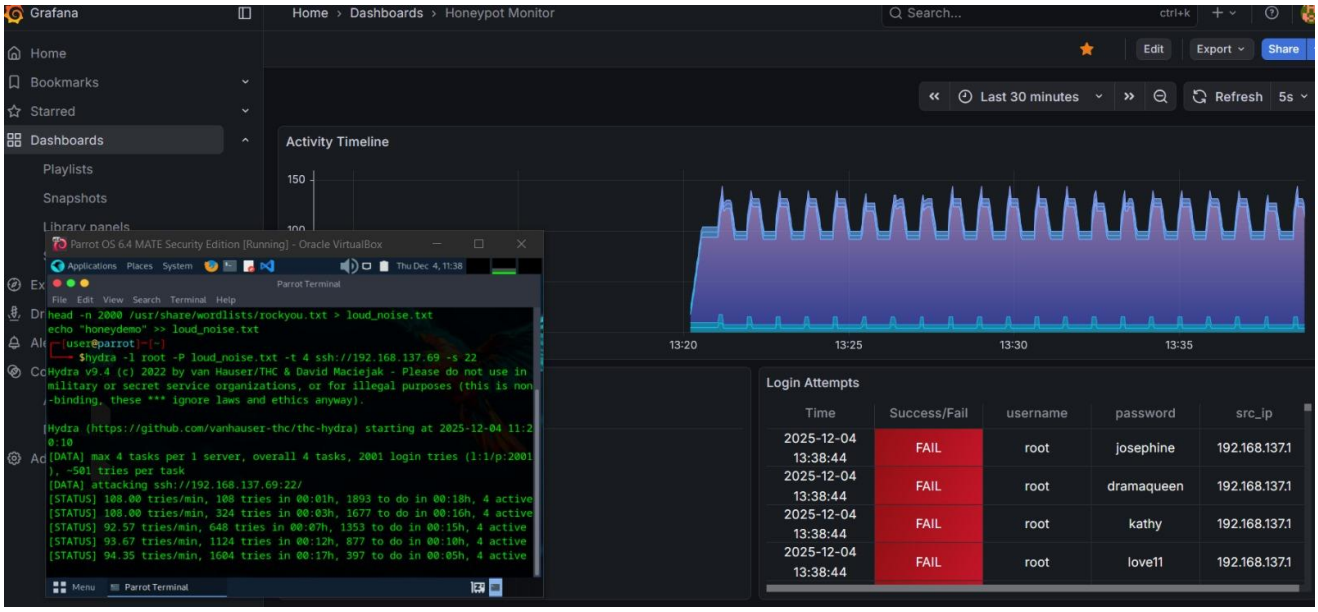



Fig 4.3.1 Grafana Dashboard displaying Cowrie logs

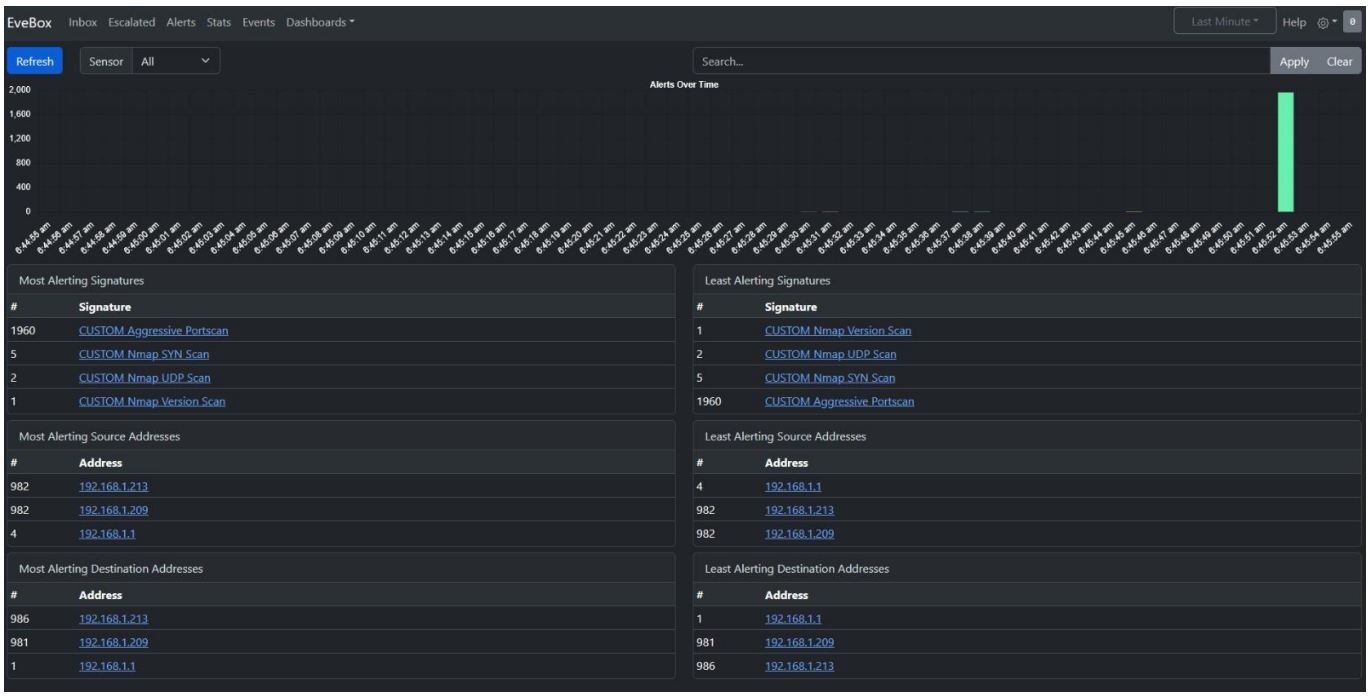



Fig 4.3.2 Eve Box displaying port scanning from custom scripts

## 4.4 Supporting Infrastructure

Several supporting components helped manage the project:

• A Git repository hosted on GitHub (**HoneyPot-SavoniaUAS**) storing configuration files, the Red Team scripts (portscanner.py, testcow.py, console.py), blue-team materials, web-server content and documentation

• Cloud-based collaboration tools from Savonia UAS for sharing screenshots, whiteboard photos and report drafts

• Static demo websites for fictional companies such as “NordEdge Technologies” and “ArcticEdge Systems”, hosted on the NGINX web server to make the network look more like a small production environment

Github repository : https://github.com/eurusfox-24/HoneyPot-SavoniaUAS.git

# 5 SECURITY CONSIDERATIONS AND THREAT LANDSCAPE

Many IoT devices still rely on legacy communication protocols that lack modern encryption and authentication, making them easy targets for exploitation. Manufacturers often ship devices with default usernames and passwords such as “admin/admin”, which users rarely change. These weak credentials can be abused using brute-force and dictionary-based attacks or more advanced techniques such as rainbow-table and birthday attacks.

If an attacker successfully exploits these weaknesses, they can infiltrate networks, steal sensitive data and compromise entire systems. Our honeypot lab highlights these issues in a safe way: Telnet and other services are intentionally exposed only inside an isolated subnet, and all activity is monitored.

The lessons for real-world IoT deployments include:

- Avoid exposing legacy protocols such as Telnet to untrusted networks.
- Change default passwords immediately and enforce strong credential policies.
- Place IoT devices and gateways in segmented network zones with firewalls.
- Enable logging and regular review of events from edge devices.

# 6 FUTURE IMPLEMENTATION

In the future, the lab can be expanded in several ways. First, new IoT protocols such as MQTT, Modbus/TCP or CoAP could be added to the honeypot so that we can study attacks against more realistic industrial and smart-building systems, not only SSH, Telnet, and Web services. Second, the monitoring stack could be developed further by adding more Grafana dashboards, refining Suricata rules, and possibly forwarding all logs to a central SIEM for long-term analysis and alerting. Finally, the current Docker-based setup could be packaged and documented so that identical honeypot kits can be easily deployed for other courses or partner organizations -turning this project into a reusable teaching and research platform for IoT security.

# 7 CONTRIBUTIONS

**Min Set Ko - Project Lead, OpenWrt Implementation**

**Md Mohiuddin Mollah - Documentation Lead, Honeypot Implementation Support**

**Arsenii Marchenko - Adversarial Simulation & Product Assembly**

**Margarita Filenko – 3d Modelling & Hardware Enclosure Design**

**Joonas Tahvanainen – Honeypot & IDS Implementation, Detection, and Analysis**

**Rupashri Das - Web Server Implementation**

# 8 CONCLUSION

In this project we designed and implemented a low-interaction IoT honeypot lab using four Raspberry Pi 5 devices. The environment consisted of an OpenWrt-based router, a Cowrie SSH/Telnet honeypot, an attacker machine running Parrot OS, and a NGINX web server hosting both static demo sites and a deliberately vulnerable PHP page. Together these devices formed a small but realistic IoT-style network that we could fully control inside a custom enclosure.

We extended the basic honeypot with a monitoring and detection stack that reflects how modern organizations handle security data. Cowrie's logs were collected by Promtail, stored in Loki and visualized in Grafana dashboards, while Suricata IDS analyzed network traffic and sent alerts to the EveBox web interface. This gave the blue team two perspectives on every attack: detailed textual log entries and higher-level dashboards that summarized scanning, brute-force attempts and web-application activity over time.

Through the red-team vs. blue-team exercise, we saw how typical attacks against exposed management services and web applications look in practice. Using Nmap and the custom scripts `portscanner.py`, `testcow.py` and `console.py`, the red team discovered open ports, performed controlled brute-force attempts against Cowrie's SSH/Telnet service and triggered a proof-of-concept exploit against the vulnerable PHP page. The resulting events were clearly visible in Cowrie logs, Suricata alerts, Eve Box views, and Grafana graphs, which helped the blue team understand the full attack chain from reconnaissance to exploitation.

All testing was carried out completely offline in our own isolated subnet. The honeypot environment was never connected to the public internet, and the subnet was separated from the Savonia network so that our experiments could not interfere with other devices. This design allowed us to experiment with offensive tools and misconfigured services in a safe and responsible way while still reflecting realistic attacker behaviors.

The project also revealed some limitations. Only a limited set of services and attack scenarios were implemented, and the environment was exercised with a single attacking host and small wordlists. Some planned components, such as additional vulnerable services and more advanced automation of Suricata rules, remain as future work. In the next iteration, the lab could be extended with richer scenarios, more diverse attack tools, and additional visualizations.

Despite these limitations, the project achieved its main goals. We built a reusable teaching lab, learned how to configure routers, honeypots, web servers and IDS tools on Raspberry Pi hardware, and gained hands-on experience with both attacking and defending IoT-style networks. Most importantly, we developed a much clearer understanding of how honeypots and monitoring stacks like Loki, Grafana, Suricata and EveBox can be combined to support practical IoT security in real environments.

# 9 REFERENCES

Artificial intelligence has been used in the work as follows:

ChatGPT 2024. OpenAI. GPT-3.5. Accessed for language check, May 2024. https://chat.openai.com.

us-17-Luo-Iotcandyjar-Towards-An-Intelligent-Interaction-Honeypot-For-IoT-Devices-wp.pdf

Lance Spitzner. (2003). *Honeypots: Tracking Hackers*. Addison-Wesley. ISBN: 9780321108951.

Top IoT Device Vulnerabilities: How To Secure IoT Devices | Fortinet

Top 10 IoT Devices Vulnerability and Attack Vectors - GeeksforGeeksIoT Vulnerabilities and How to Mitigate Them | Sternum IoT

https://github.com/grafana/loki.git

https://github.com/cowrie/cowrie.git

Microsoft. (2025). *Copilot* [Large language model].